

Sprawozdanie z Laboratorium nr 2

Przedmiot: Integracja Systemów Informatycznych

Temat: Konteneryzacja aplikacji Django za pomocą Dockera **Data:** 2026-06-01 **Student:** Aleksandr Vasiljev

1. Cel laboratorium

Przygotowanie aplikacji do pracy w środowisku izolowanym przy użyciu Dockera. Konfiguracja Dockerfile oraz docker-compose dla aplikacji Python (Django).

2. Realizacja zadań

Wszystkie zmiany były wykonywane w osobnej gałęzi `feature/dockerization`.

Zadanie 1: Przygotowanie środowiska

- **Opis działań:** Stworzono nową gałąź: `feature/dockerization`. Dodano plik `.dockerignore`, aby uniknąć kopiowania zbędnych plików.
- **Zastosowane komendy Git:**

```
git checkout -b feature/dockerization
```

Zadanie 2: Tworzenie Dockerfile

- **Opis działań:** Przygotowano plik Dockerfile oraz zbudowano obraz.

```
FROM python:3.11-slim

WORKDIR /LAB1

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

- **Zastosowane komendy Git:**

```
git add .  
git commit -m "Add Dockerfile and .dockerignore"
```

Zadanie 3: Orkiestracja z Docker Compose

- **Opis działań:** Stworzono plik docker-compose.yml zawierający serwis web oraz db. Skonfigurowano zmienne środowiskowe dla połączenia z bazą danych. Uruchumiono cały stos: `docker-compose up`.

```
services:  
  web:  
    build: .  
    ports:  
      - "8000:8000"  
    depends_on:  
      - db  
    environment:  
      DB_NAME: mydb  
      DB_USER: postgres  
      DB_PASSWORD: postgres  
      DB_HOST: db  
  
  db:  
    image: postgres:16  
    restart: always  
    environment:  
      POSTGRES_DB: mydb  
      POSTGRES_USER: postgres  
      POSTGRES_PASSWORD: postgres  
    ports:  
      - "5432:5432"  
    volumes:  
      - postgres_data:/var/lib/postgresql/data  
  
volumes:  
  postgres_data:
```

- **Zastosowane komendy Git:**

```
git add .  
git commit -m "Add docker-compose for app and database orchestration"
```

Zrzut ekranu – Uruchumiono cały stos

```
PS C:\Users\Aleks\Desktop\unik\ISI\ISI\LAB1> docker-compose up --build
#1 [internal] load local bake definitions
#1 reading from stdin 529B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 245B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 0.6s
```

Zadanie 4: Zarządzanie kontenerami i inspekcja

- **Opis działań:** Sprawdzono działające kontenery: `docker ps`. Przejrzano logi aplikacji: `docker-compose logs -f web`. Przejrzno wnętrza kontenera: `docker-compose exec web bash`. Sprawdzono zużycie zasobów: `docker stats`. Po uruchomieniu kontenerów wykonano migracje bazy danych wewnątrz kontenera aplikacji Django. Dzięki temu utworzone zostały wszystkie wymagane tabele w bazie PostgreSQL.

Zrzut ekranu – Kontenery

```
PS C:\Users\Aleks\Desktop\unik\ISI\ISI\LAB1> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
8f8ee18fb80a	lab1-web	"python manage.py ru..."	23 minutes ago	Up 23 minutes	0.0.0.0:8000
		lab1-web-1			
13baa2b7d60f	postgres:16	"docker-entrypoint.s..."	23 minutes ago	Up 23 minutes	0.0.0.0:5432
		lab1-db-1			

```
PS C:\Users\Aleks\Desktop\unik\ISI\ISI\LAB1>
```

Zrzut ekranu – Logi

```
PS C:\Users\Aleks\Desktop\unik\ISI\ISI\LAB1> docker-compose logs -f web
web-1 | Watching for file changes with StatReloader
```

Zrzut ekranu – Wnętrze kontenera

```
PS C:\Users\Aleks\Desktop\unik\ISI\ISI\LAB1> docker-compose exec web bash
root@8f8ee18fb80a:/LAB1#
```

Zrzut ekranu – Zużycie zasobów

```
CONTAINER ID   NAME          CPU %       MEM USAGE / LIMIT   MEM %       NET I/O       BLOCK I/O
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
8f8ee18fb80a	lab1-web-1	1.32%	117.1MiB / 15.53GiB	0.74%	66.9kB / 279kB	41.7MB / 1.5
13baa2b7d60f	lab1-db-1	0.00%	27.59MiB / 15.53GiB	0.17%	58kB / 40.2kB	19.1MB / 2.0

```
1MB  7
```

Wnioski

Podczas laboratorium poznano proces konteneryzacji aplikacji Django z wykorzystaniem Dockera i Docker Compose. Utworzono obraz aplikacji, skonfigurowano połączenie z bazą danych PostgreSQL oraz uruchomiono cały system w odizolowanym środowisku. Nauczyłem się zarządzać kontenerami, analizować logi aplikacji oraz wykonywać migracje bazy danych wewnątrz kontenera. Docker znacząco upraszcza proces uruchamiania aplikacji na różnych środowiskach i zapewnia powtarzalność konfiguracji projektu.